Team Name : Team Jarvinder

Team Leader Name : Paras Chugh

Problem Statement : Wavefront Reconstruction and Turbulence Characterization using SH-WFS Time-Series Data — A Hybrid Physics + AI Adaptive Optics Framework

Team Members

Team Leader:

Name:Paras Chugh
College:Delhi Technological University

Team Member-1:

Name:Amanpal Singh
College:Delhi Technological University

Team Member-2:

Name: Anmol Saluja
College: Vellore Institute of Technology, Chennai Campus

Team Member-3:

Name: Aditya Kumar
College: Vellore Institute of Technology, Chennai Campus

Opportunity: Hybrid Physics + AI Adaptive Optics for SH-WFS

AI-Enhanced Pipeline

Classical AO is computationally expensive (>10 ms latency) and purely reactive. This proposal embeds three targeted AI modules:

- **MLP Reconstructor:** Replaces slow matrix inversion for wavefronts.
- **Temporal Predictor:** LSTM/GRU forecasting next wavefront state.
- **Neural Mapper:** Maps reconstructed wavefront to DM commands.

Real-Time Execution

Processes SH-WFS slope vectors to output Zernike coefficients with high precision and speed:

- **Sub-ms Inference:** Eliminates major matrix inversion bottlenecks.
- **Predictive Control:** Eliminates servo lag by forecasting frames ahead.
- **Synthetic Training:** Uses forward optical model, removing physical bench dependencies initially.

Core USP

A physics-grounded hybrid framework that elevates AO performance:

- **Physics Foundation:** Retains Zernike basis and forward optical models.
- **Modular Integration:** Upgrades can be independently validated on active pipelines.
- **True Anticipation:** Overcomes late reaction by anticipating future distortion states.

Key Features of the Proposed Solution

1. MLP Reconstructor

Maps SH-WFS slope vectors to Zernike coefficients; eliminates matrix inversion; sub-ms inference; robust to noise.

2. Temporal Compensator

LSTM/GRU/Transformer trained on wavefront time series; forecasts next-frame; reduces servo lag; enables predictive DM.

3. Neural Mapper

Maps wavefront to DM actuator strokes; learns coupling implicitly; enforces stroke constraints; faster than matrix inversion.

4. Synthetic Pipeline

Generates slope vectors from random Zernike coefficients via forward optical model; injects realistic Gaussian noise.

5. Turbulence Estimation

Computes Fried parameter (r0) and coherence time (tau0) from reconstructed wavefront statistics per frame.

6. Pipeline Output

Per-frame: wavefront phase map, Zernike coefficient vector, r0, tau0, DM actuator command map.

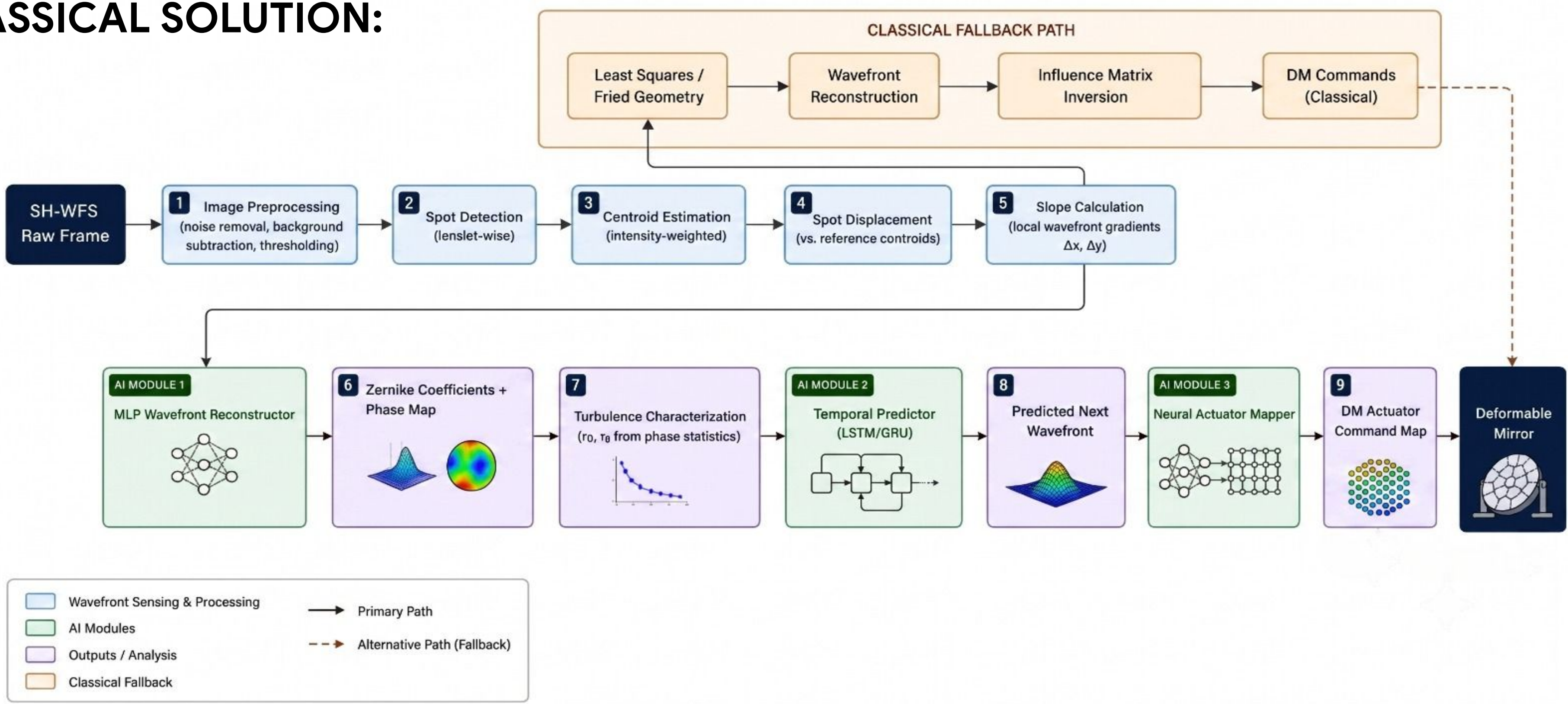
7. Modular Design

Each AI module is independently replaceable; classical fallback is always available; physics is never bypassed.

8. Real-Time Ready

MLP and GRU inference targets <1 ms on GPU; entire hybrid pipeline is designed for a 10 ms AO loop rate.

PROCESS FLOW OF THE PROPOSED SOLUTION COMPARED TO THE CLASSICAL SOLUTION:



Synthetic Training Data Generation Pipeline (AI Module 1 — MLP Wavefront Reconstructor)

01. Sample Zernike

Draw N random Zernike coefficient vectors from a Kolmogorov turbulence distribution (parameterized by r0 range).

02. Forward Optical Model

Compute theoretical SH-WFS slope vectors using lenslet geometry and aperture mask (physics forward model, no approximation).

03. Noise Injection

Add Gaussian noise to simulate detector read noise, photon noise, and centroiding error (SNR range: 10–30 dB).

04. Dataset Assembly

Map noisy slope vector (input X) to original Zernike coefficients (label Y). Dataset size: ~100k–500k samples.

05. MLP Training

Supervised regression using MSE loss on Zernike coefficients. Architecture: 3–5 fully connected layers with ReLU and batch norm.

06. Validation

Compare reconstructed wavefronts to ground truth (RMS error) and benchmark against classical least-squares.

Key Point: This approach requires no AO bench or real atmospheric data for initial model training—physics provides the ground truth.

System Architecture: Hybrid Physics + AI Adaptive Optics

01. SENSOR LAYER
Camera / Detector ⟶ Raw SH-WFS frame (2D image)

02. PREPROCESSING LAYER
Image Preprocessing: background subtraction, noise filtering ⟶ Spot Detection ⟶ Centroid Estimation ⟶ Slope Vector (2N-dim)

03. RECONSTRUCTION LAYER
AI Module 1: MLP Wavefront Reconstructor (Input: slope | Output: Zernike + phase map)
Classical bypass: Least Squares reconstructor

04. CHARACTERIZATION LAYER
Turbulence Estimator: computes r0 and tau0 from phase statistics

05. PREDICTION LAYER
AI Module 2: Temporal Predictor (LSTM / GRU)
Input: last K wavefronts | Output: predicted next wavefront (K=5–20)

06. CONTROL LAYER
AI Module 3: Neural Actuator Command Mapper (Input: wavefront | Output: DM commands)
Classical bypass: Influence matrix inversion

07. ACTUATION LAYER
Deformable Mirror Controller ⟶ DM actuation ⟶ corrected wavefront

08. VISUALIZATION / LOGGING
Real-time phase map display | Zernike coefficient spectrum | r0 / tau0 time series | Residual wavefront error monitor

Technologies Used

Programming
Python 3.10+, NumPy, SciPy

Machine Learning
PyTorch (MLP, LSTM, GRU, Transformer), scikit-learn (baseline models)

Optics / AO Math
Custom forward optical model (Zernike, lenslet, influence matrix), HCIPy / POPPY

Image Processing
OpenCV (centroid estimation, thresholding, background subtraction), scikit-image

Scientific Computing
SciPy (sparse linear solvers, FFT, statistics), Astropy (optional FITS I/O)

Optimization
Adam optimizer (MLP training), learning rate schedulers, early stopping

Visualization
Matplotlib (phase maps, Zernike spectra, plots), real-time plotting via PyQtGraph

Data Pipeline
NumPy arrays for synthetic dataset; HDF5 (h5py) for large dataset storage

Hardware Target
NVIDIA GPU (CUDA) for training and inference; CPU viable for smaller models

Estimated Implementation Cost

Software Development

All code is open-source (Python, PyTorch, NumPy). Zero software licensing cost.

Compute — Training

Google Colab Pro / Kaggle GPU (~INR 0–1,000/mo). A single T4/V100 session trains models within 12–24 hours.

Compute — Inference

Any mid-range laptop GPU or CPU is sufficient for real-time 10 ms inference. Cost: INR 0 (using existing hardware).

Data Generation

Fully synthetic — no cost. The forward optical model generates all training data programmatically.

AO Simulation

HCIPy / POPPY — free, open-source environments used for optical simulation and validation.

Deployment (Post-Hack)

Hardware cost depends on target AO bench. Software remains open source; edge GPU (Jetson AGX) is viable.

Estimated Total Cost for Hackathon Prototype: INR 0 – 2,000 (cloud GPU hours only, if local hardware is unavailable).

BHARATIYA ANTARIKSH HACKATHON 2026

THANK YOU